



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

INFORME DE LABORATORIO
“Máquina Expendedora”

Curso: Programación II

Docente: Ing. Breno Luque Zuñiga

Tarqui Ramos, Sebastián Emir Andree (2022075482)

Tacna – Perú
2026



INDICE

I. INFORMACIÓN GENERAL	3
Objetivos	3
Equipos, materiales, programas y recursos utilizados.....	3
II. MARCO TEORICO	3
III. PROCEDIMIENTO	4
Controles utilizados en el formulario	4
Pasos de implementación	5
IV. ANALISIS E INTERPRETACION DE RESULTADOS.....	10
¿Qué indican los resultados?.....	10
¿Qué se ha encontrado?.....	10
V. CUESTIONARIO	10
CONCLUSIONES.....	11



INFORME DE LABORATORIO

Máquina Expendedora

I. INFORMACIÓN GENERAL

Objetivos

- Simular el funcionamiento de una máquina expendedora real mediante una aplicación de escritorio en C# Windows Forms.
- Implementar lógica de inserción de monedas, selección de productos por código y cálculo de vuelto.
- Gestionar el stock de productos y registrar las ventas realizadas durante la sesión.
- Generar un reporte de ventas filtrable por día, mes y año.
- Aplicar el uso de clases, listas genéricas y eventos en C# para estructurar una solución orientada a objetos.

Equipos, materiales, programas y recursos utilizados

- Computadora con Windows 10/11
- Visual Studio 2022
- Lenguaje C# (.NET Framework / .NET 6+)
- Windows Forms (WinForms)
- Clases personalizadas: Producto, VentaRegistro
- DataGridView, Panel, Button, TextBox, Label

II. MARCO TEORICO

Windows Forms

Windows Forms es una plataforma de desarrollo de interfaces gráficas para escritorio incluida en el ecosistema .NET de Microsoft. Permite crear ventanas, formularios y controles visuales de manera rápida, usando el diseñador visual de Visual Studio o código manual.

Clases y Programación Orientada a Objetos

Se utilizaron dos clases principales: Producto, que encapsula los atributos de cada ítem de la máquina (código, nombre, precio, stock), y VentaRegistro, que guarda los datos de cada transacción (fecha, producto vendido, monto pagado y vuelto entregado).

Listas Genéricas (List<T>)

Las colecciones List<Producto> y List<VentaRegistro> permiten almacenar y manipular dinámicamente los datos en memoria durante la ejecución del programa, sin necesidad de una base de datos.

DataGridView

El control DataGridView permite mostrar datos tabulares de forma visual. En este proyecto se usó para listar los productos disponibles con su código, nombre, precio y stock, actualizándose automáticamente tras cada venta.



Eventos de botones

Cada botón del teclado virtual y de monedas está enlazado a un método controlador de eventos (Event Handler) que ejecuta la lógica correspondiente: acumular crédito, escribir dígitos en el TextBox o procesar la compra.

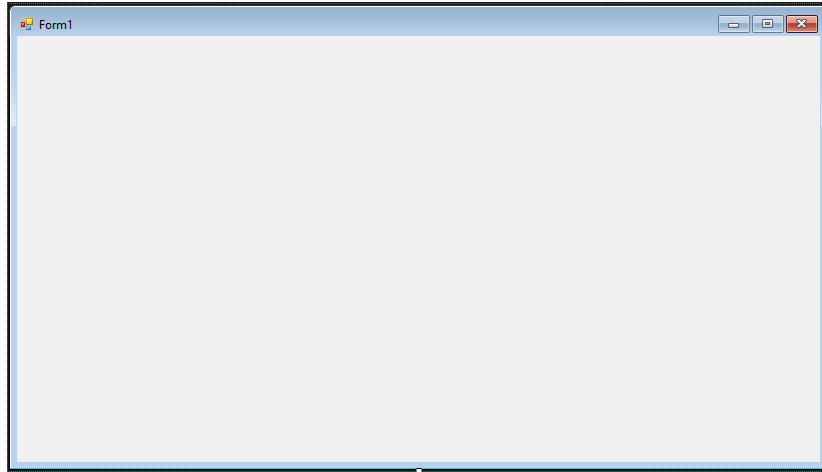
III. PROCEDIMIENTO

Controles utilizados en el formulario

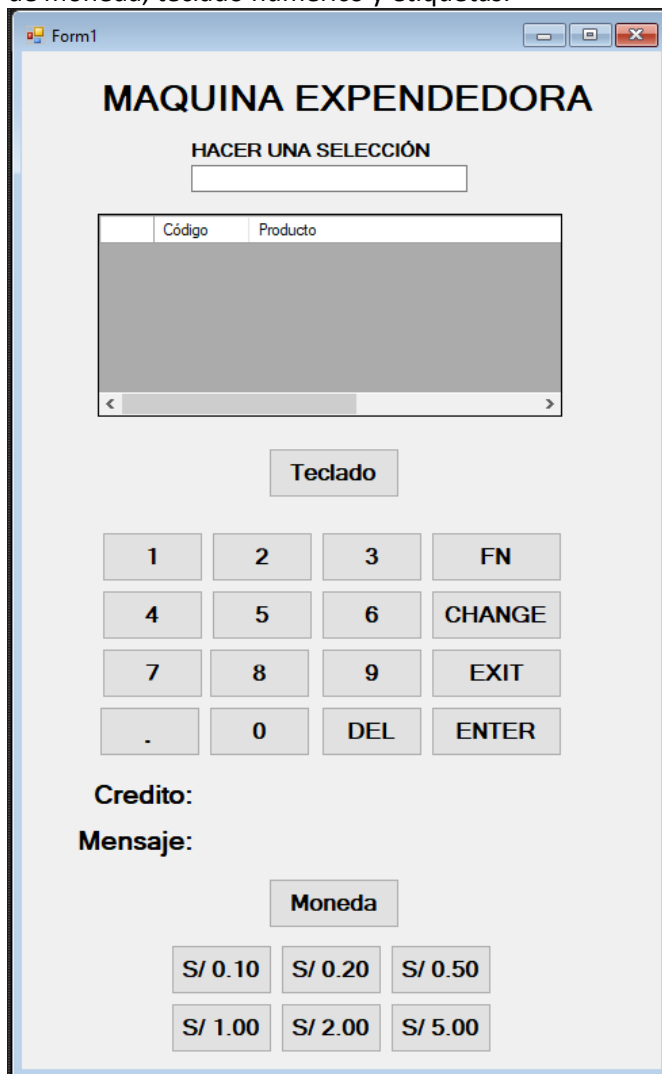
Control	Nombre	Descripción
Label	lblTitulo	Muestra el título de la máquina expendedora
Label	lblCredito	Muestra el crédito acumulado en tiempo real
Label	lblMensaje	Muestra mensajes de estado, errores y vuelto
TextBox	txtCodigo	Muestra el código ingresado por el teclado virtual
DataGridView	dgvProductos	Lista todos los productos con código, nombre, precio y stock
Button x6	btnM010 ... btnM500	Botones para insertar monedas de S/0.10 a S/5.00
Button x10	btn0 ... btn9, btnPunto	Teclado numérico para ingresar el código del producto
Button	btnDEL	Borra el último dígito ingresado
Button	btnENTER	Confirma la selección y procesa la venta
Button	btnCHANGE	Devuelve el crédito sin realizar compra
Button	btnEXIT	Limpia el código ingresado
Button	btnFN	Abre el formulario de reporte de ventas
Panel	panelDerecho	Agrupar los controles del teclado y monedas

Pasos de implementación

1. Se creó el proyecto Windows Forms con el namespace.



2. Se diseñó el formulario Form1 en el Designer con todos los controles: DataGridView, botones de moneda, teclado numérico y etiquetas.

A screenshot of the designed Windows Forms application window titled 'Form1'. The form has a title 'MAQUINA EXPENDEDORA' and a subtitle 'HACER UNA SELECCIÓN'. Below the subtitle is a text box. Underneath is a DataGridView with two columns: 'Código' and 'Producto'. Below the DataGridView is a button labeled 'Teclado'. Below the 'Teclado' button is a numeric keypad with buttons for digits 1-9, 0, a decimal point, and function keys 'FN', 'CHANGE', 'EXIT', 'ENTER', 'DEL'. Below the keypad are labels 'Credito:' and 'Mensaje:'. At the bottom is a button labeled 'Moneda'. Below the 'Moneda' button are six buttons for coin denominations: 'S/ 0.10', 'S/ 0.20', 'S/ 0.50', 'S/ 1.00', 'S/ 2.00', and 'S/ 5.00'.

3. Se crearon las clases Producto.cs y VentaRegistro.cs para encapsular los datos.

```
9 public class Producto
10 {
11     public string Codigo { get; set; }
12     - referencias
13     public string Nombre { get; set; }
14     6 referencias
15     public double Precio { get; set; }
16     5 referencias
17     public int Stock { get; set; }
18
19     30 referencias
20     public Producto(string codigo, string nombre, double precio, int stock)
21     {
22         Codigo = codigo;
23         Nombre = nombre;
24         Precio = precio;
25         Stock = stock;
26     }
27 }
28
29 public class VentaRegistro
30 {
31     public DateTime Fecha { get; set; }
32     public string Codigo { get; set; }
33     public string Nombre { get; set; }
34     public double Precio { get; set; }
35     public double Pagado { get; set; }
36     public double Vuelto { get; set; }
37
38     public VentaRegistro(string codigo, string nombre, double precio, double pagado, double vuelto)
39     {
40         Fecha = DateTime.Now;
41         Codigo = codigo;
42         Nombre = nombre;
43         Precio = precio;
44         Pagado = pagado;
45         Vuelto = vuelto;
46     }
47 }
```

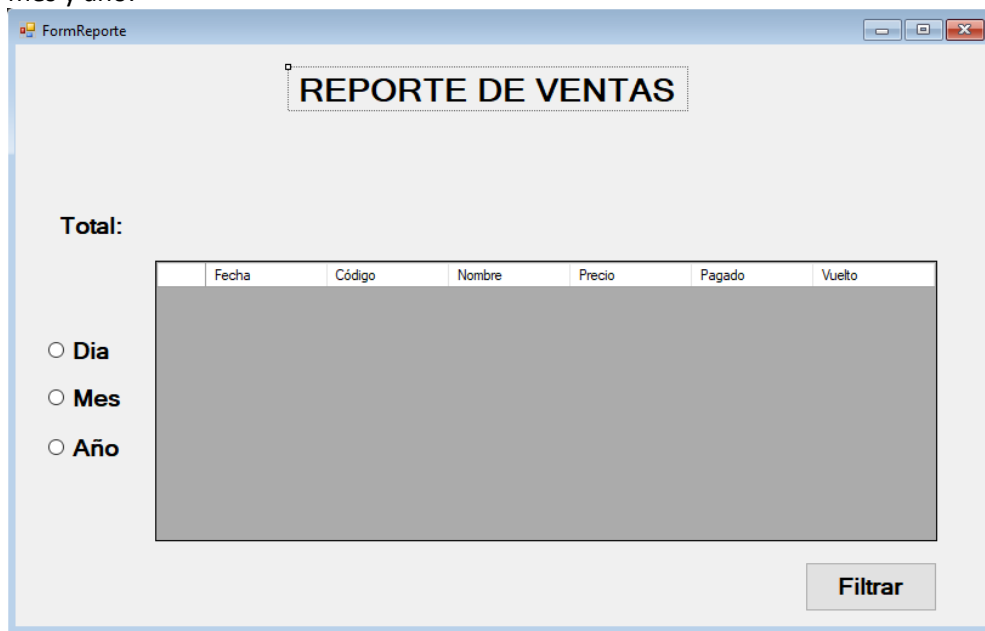
4. En Form1.cs se implementó la lógica: carga de 30 productos en 5 filas de 6, eventos de monedas, teclado numérico, ENTER (procesamiento de venta), CHANGE (devolución de crédito) y EXIT (limpieza).

```
25 private void CargarProductos()
26 {
27     // Fila 1 - Snacks
28     productos.Add(new Producto("011", "Chifles", 2.50, 5));
29     productos.Add(new Producto("012", "Doritos", 3.00, 5));
30     productos.Add(new Producto("013", "Pringles", 5.00, 5));
31     productos.Add(new Producto("014", "Galletas", 2.00, 5));
32     productos.Add(new Producto("015", "Mani", 1.50, 5));
33     productos.Add(new Producto("016", "Chicles", 0.50, 5));
34     // Fila 2 - Dulces
35     productos.Add(new Producto("021", "Chocolate", 3.50, 5));
36     productos.Add(new Producto("022", "Galleta Choc", 2.50, 5));
37     productos.Add(new Producto("023", "Caramelos", 1.00, 5));
38     productos.Add(new Producto("024", "Wafer", 1.50, 5));
39     productos.Add(new Producto("025", "Bizcocho", 2.00, 5));
40     productos.Add(new Producto("026", "Turrón", 1.00, 5));
41     // Fila 3 - Bebidas
42     productos.Add(new Producto("031", "Agua 500ml", 1.50, 5));
43     productos.Add(new Producto("032", "Jugo Naranja", 3.50, 5));
44     productos.Add(new Producto("033", "Jugo Mango", 3.50, 5));
45     productos.Add(new Producto("034", "Té Helado", 3.00, 5));
46     productos.Add(new Producto("035", "Energizante", 5.00, 5));
47     productos.Add(new Producto("036", "Limonada", 3.00, 5));
48     // Fila 4 - Gaseosas
49     productos.Add(new Producto("041", "Coca-Cola", 3.50, 5));
50     productos.Add(new Producto("042", "Pepsi", 3.50, 5));
51     productos.Add(new Producto("043", "Sprite", 3.50, 5));
52     productos.Add(new Producto("044", "Fanta", 3.50, 5));
53     productos.Add(new Producto("045", "Inca Kola", 3.50, 5));
54     productos.Add(new Producto("046", "Guaraná", 3.00, 5));
55     // Fila 5 - Latas
56     productos.Add(new Producto("051", "Red Bull", 6.00, 5));
57     productos.Add(new Producto("052", "Monster", 6.00, 5));
58     productos.Add(new Producto("053", "Cerveza s/a", 5.00, 5));
59     productos.Add(new Producto("054", "Pepsi Lata", 3.50, 5));
60     productos.Add(new Producto("055", "Coca Lata", 3.50, 5));
61     productos.Add(new Producto("056", "Seven Up", 3.00, 5));
62 }
```

5. Se configuraron las columnas del DataGridView por código al iniciar el formulario.

```
64     private void ActualizarGrilla()
65     {
66         dgvProductos.Rows.Clear();
67         foreach (var p in productos)
68         {
69             var fila = dgvProductos.Rows.Add();
70             dgvProductos.Rows[fila].Cells["colCodigo"].Value = p.Codigo;
71             dgvProductos.Rows[fila].Cells["colNombre"].Value = p.Nombre;
72             dgvProductos.Rows[fila].Cells["colPrecio"].Value = p.Precio.ToString("F2");
73             dgvProductos.Rows[fila].Cells["colStock"].Value = p.Stock;
74
75             if (p.Stock == 0)
76                 dgvProductos.Rows[fila].DefaultCellStyle.ForeColor = Color.Red;
77         }
78     }
```

6. Se creó el FormReporte.cs con un DataGridView y RadioButtons para filtrar ventas por día, mes y año.



Fecha	Código	Nombre	Precio	Pagado	Vuelto
-------	--------	--------	--------	--------	--------

```
13 public partial class FormReporte : Form
14 {
15     private List<VentaRegistro> ventas;
16     1 referencia
17     public FormReporte(List<VentaRegistro> ventas)
18     {
19         InitializeComponent();
20         this.ventas = ventas;
21         rbDia.Checked = true;
22         Filtrar();
23     }
24     1 referencia
25     private void btnFiltrar_Click(object sender, EventArgs e)
26     {
27         Filtrar();
28     }
29     private void Filtrar()
30     {
31         List<VentaRegistro> filtradas;
32         DateTime hoy = DateTime.Now;
33
34         if (rbDia.Checked)
35             filtradas = ventas.Where(v => v.Fecha.Date == hoy.Date).ToList();
36         else if (rbMes.Checked)
37             filtradas = ventas.Where(v => v.Fecha.Month == hoy.Month && v.Fecha.Year == hoy.Year).ToList();
38         else
39             filtradas = ventas.Where(v => v.Fecha.Year == hoy.Year).ToList();
40
41         dgvReporte.Rows.Clear();
42         double totalGeneral = 0;
43
44         foreach (var v in filtradas)
45         {
46             dgvReporte.Rows.Add(
47                 v.Fecha.ToString("dd/MM/yyyy HH:mm"),
48                 v.Codigo,
49                 v.Nombre,
50                 v.Precio.ToString("F2"),
51                 v.Pagado.ToString("F2"),
52                 v.Vuelto.ToString("F2")
53             );
54             totalGeneral += v.Precio;
55         }
56
57         lblTotal.Text = $"Total vendido: S/ {totalGeneral:F2} | Ventas: {filtradas.Count}";
58     }
```

7. Se asignaron los eventos de cada botón en el Designer apuntando a los métodos correspondientes en el código.

```
85 private void BtnMoneda_Click(object sender, EventArgs e)
86 {
87     Button btn = (Button)sender;
88     double valor = 0;
89
90     if (btn.Name == "btnM010") valor = 0.10;
91     else if (btn.Name == "btnM020") valor = 0.20;
92     else if (btn.Name == "btnM050") valor = 0.50;
93     else if (btn.Name == "btnM100") valor = 1.00;
94     else if (btn.Name == "btnM200") valor = 2.00;
95     else if (btn.Name == "btnM500") valor = 5.00;
96
97     credito = Math.Round(credito + valor, 2);
98     lblCredito.Text = "S/ " + credito.ToString("F2");
99     lblMensaje.ForeColor = Color.Yellow;
100     lblMensaje.Text = "Moneda de S/ " + valor.ToString("F2") + " ingresada";
101 }
```




```
151 private void BtnCHANGE_Click(object sender, EventArgs e)
152 {
153     if (credito > 0)
154     {
155         MostrarMensaje($"Devuelto: S/ {credito:F2}", Color.Cyan);
156         credito = 0;
157         lblCredito.Text = "S/ 0.00";
158     }
159     else
160     {
161         MostrarMensaje("No hay crédito para devolver", Color.Yellow);
162     }
163 }
164
165 1 referencia
166 private void BtnEXIT_Click(object sender, EventArgs e)
167 {
168     txtCodigo.Text = "";
169     lblMensaje.Text = "";
170 }
171
172 1 referencia
173 private void BtnFN_Click(object sender, EventArgs e)
174 {
175     FormReporte fr = new FormReporte(ventas);
176     fr.ShowDialog();
177 }
```



IV. ANALISIS E INTERPRETACION DE RESULTADOS

¿Qué indican los resultados?

El sistema responde correctamente al flujo de una máquina expendedora real: acumula monedas, valida el crédito antes de despachar, descuenta el stock y entrega vuelto. Los mensajes de estado informan al usuario en todo momento qué ocurrió con su operación.

¿Qué se ha encontrado?

- Al intentar agregar filas al DataGridView sin columnas definidas previamente se produjo una `InvalidOperationException`. Se solucionó llamando a `ConfigurarColumnas()` antes de cargar datos.
- El manejo de decimales en el crédito requirió `Math.Round` con 2 decimales para evitar errores de punto flotante (ej: $0.10 + 0.20 = 0.300000000004$).
- La lista de ventas se mantiene en memoria durante la sesión; al cerrar el programa los datos se pierden, lo que es una limitación para un sistema real.
- El reporte filtra correctamente por día, mes y año usando LINQ sobre la lista de `VentaRegistro`.

V. CUESTIONARIO

1. ¿Por qué se usaron clases separadas (`Producto` y `VentaRegistro`) en lugar de manejar todo en `Form1`?

Para separar responsabilidades. Cada clase tiene un propósito claro: `Producto` modela el inventario y `VentaRegistro` modela las transacciones. Esto hace el código más ordenado y fácil de mantener.

2. ¿Qué pasaría si no se usa `Math.Round` al acumular el crédito?

Se acumularían errores de punto flotante. Por ejemplo, insertar `S/0.10` tres veces daría `0.30000000000000004` en lugar de `0.30`, lo que causaría problemas al comparar con el precio del producto.

3. ¿Cómo se podría persistir el historial de ventas entre sesiones?

Guardando los registros en un archivo (JSON, XML, CSV) o en una base de datos SQL Server al cerrar el formulario y cargándolos al iniciar.

4. ¿Para qué sirve el botón `CHANGE` en una máquina expendedora real?

Devuelve el dinero insertado cuando el usuario decide no comprar nada. En el sistema, resetea el crédito a cero y muestra el monto devuelto.

5. ¿Qué ventaja tiene usar `List<T>` frente a un array para los productos?

`List<T>` tiene tamaño dinámico, permite agregar y buscar elementos fácilmente con métodos como `Add()` y `Find()`, sin necesidad de definir el tamaño de antemano.



CONCLUSIONES

- Se logró implementar correctamente la simulación de una máquina expendedora con todas las funciones básicas: inserción de monedas, selección por código, despacho de producto, vuelto y reporte de ventas.
- El uso de clases separadas (Producto y VentaRegistro) demostró ser clave para mantener el código organizado y evitar que Form1.cs se volviera difícil de leer.
- Los errores de punto flotante son un problema real al trabajar con dinero en C#; Math.Round es una solución sencilla y efectiva.
- Windows Forms permite construir aplicaciones de escritorio funcionales de forma rápida, aunque para un sistema de producción real se necesitaría persistencia de datos y mayor seguridad en la lógica de transacciones.
- El proyecto reforzó el manejo de eventos, listas genéricas, LINQ y la separación entre diseño (Designer) y lógica (code-behind) en aplicaciones WinForms.